

Caso pedagógico 04 — Config/validação e segurança básica (auth/users/events/registrations)

1) Objetivo pedagógico

Este caso foi escrito para ensinar duas competências essenciais em projetos reais:

1. **Confiabilidade de configuração:** o serviço não deve iniciar “meio certo”. Se variáveis críticas estiverem ausentes ou inválidas, o *bootstrap* deve falhar cedo (fail-fast).
2. **Segurança mínima por padrão:** todo serviço HTTP exposto deve iniciar com um conjunto básico de proteções (cabeçalhos, CORS controlado, limitação de requisições), reduzindo abuso accidental e riscos óbvios.

O caso se aplica aos quatro serviços NestJS do projeto (**auth**, **users**, **events**, **registrations**) e foi implementado de forma consistente em todos eles.

2) Problema identificado

Havia um padrão recorrente nos serviços:

- **Uso direto de** `process.env` **com** `||` e sem validação de variáveis.
 - Ex.: `const port = process.env.PORT || 3010;`
 - Isso mascara erros: uma env vazia, errada ou faltante pode cair em defaults não intencionais.
 - **Ausência de validação formal** das variáveis de ambiente (tokens, DB, URLs internas).
 - O serviço “subia”, mas falhava depois em runtime (pior para operação e depuração).
 - **Bootstrap sem segurança mínima** (sem `helmet`, CORS parametrizado e rate limit), abrindo margem para:
 - comportamentos inseguros por padrão;
 - abuso de requisições e saturação.
-

3) Impacto potencial

1. **Subida com configuração incompleta/errada**
2. Tokens de serviço/JWT ausentes, DB com credenciais erradas, URL interna incorreta.
3. **Falhas de integração difíceis de diagnosticar**

4. O serviço inicia e só quebra quando executa a primeira query ou chama um serviço dependente.

5. Superfície de ataque e abuso

6. Sem cabeçalhos de segurança, sem CORS controlado e sem rate limit, aumenta o risco de exploração e saturação.

4) Solução fundamentada

A solução foi composta por quatro pilares (todos aplicados de forma padronizada nos serviços):

Pilar A — `ConfigModule` global + validação com schema

- Adotar `ConfigModule` global no NestJS.
- Validar envs com um **schema Joi**, garantindo que:
 - variáveis críticas existam;
 - tipos estejam corretos (número, string, URL);
 - restrições mínimas sejam cumpridas (porta válida, limites de rate limit etc.).

Por que isso importa? - Evita “subir errado”. - Erros aparecem imediatamente no console, de forma clara.

Pilar B — Ler envs via `ConfigService`

- Eliminar acesso disperso a `process.env`.
- Centralizar leitura com `ConfigService`, permitindo:
 - padronização;
 - testes mais fáceis;
 - refatoração segura.

Pilar C — Segurança básica no bootstrap

Aplicar em `main.ts`: - `helmet` (cabeçalhos de segurança) - `enableCors` parametrizado por env - rate limiting simples (in-memory) como camada mínima

Nota pedagógica: rate limit in-memory é adequado para aprendizado e dev, mas não é suficiente para múltiplas réplicas em produção.

Pilar D — `@nestjs/throttler` (guard) como padrão de rate limit

- Adotar `ThrottlerModule` + `ThrottlerGuard` para padronizar limitação por janela.
- Usar `forRootAsync` com `ConfigService`.

Por que usar Throttler? - Integra nativamente ao NestJS, com guard e políticas por rota. - Permite evolução futura para storage externo (ex.: Redis) sem reescrever tudo.

5) Novas dependências: o que são e por que usamos

joi

- Biblioteca de validação de dados.
- No nosso projeto, é usada para:
 - validar variáveis de ambiente no startup;
 - garantir tipos/formatos mínimos.

Benefício: falha rápida e mensagem de erro legível para quem está configurando o sistema.

helmet

- Middleware que adiciona **cabeçalhos de segurança** HTTP.
- Ajuda a mitigar riscos comuns (por exemplo, comportamento inseguro padrão do navegador em certos cenários).

Benefício: segurança “mínima por padrão” sem custo alto de implementação.

@nestjs/throttler

- Módulo oficial do NestJS para **rate limiting**.
- Fornece guard, integração com DI e configurações por rota/escopo.

Benefício: reduz saturação e abuso; melhora previsibilidade operacional.

Observação importante: o `package-lock` não foi atualizado no momento da mudança. Em projetos reais, recomenda-se sempre manter lockfile consistente com as dependências.

6) Implementações (delta aplicado) — visão do aluno

As mudanças foram aplicadas em todos os quatro serviços, seguindo o mesmo padrão:

auth-service

- `app.module.ts` e `main.ts`:
- `ConfigModule` global com Joi (`PORT`, `DB_`, `JWTs`, `SERVICE_TOKEN`, `USERS_API_URL`, `RATE_LIMIT_`).
- `ThrottlerModule` + guard.
- TypeORM assíncrono com `ConfigService`.
- `helmet`, CORS e rate limit.

users-service

- `app.module.ts` e `main.ts`:
- `ConfigModule` global (`PORT`, `DB_`, `JWTs`, `SERVICE_TOKEN`, `RATE_LIMIT_`).
- Throttler + guard.
- TypeORM e JWT via `ConfigService`.
- `helmet`, CORS e rate limit.

events-service

- `app.module.ts` e `main.ts`:
- `ConfigModule` global (`PORT`, `DB_`, `JWTs`, `SERVICE_TOKEN`, `RATE_LIMIT_`).
- Throttler + guard.
- TypeORM e JWT via `ConfigService`.
- `helmet`, CORS e rate limit.

registrations-service

- `registrations.module.ts` e `main.ts`:
- `ConfigModule` global (`PORT`, `DB_`, `SERVICE_TOKEN`, `RATE_LIMIT_`, `DEV_AUTO_AUTH`).
- Throttler + guard.
- `helmet`, CORS e rate limit.

7) Complemento operacional (pertinente para alunos)

A seguir estão pontos que valem explicitamente para quem está aprendendo a executar e operar o projeto.

7.1) Atenção com `@nestjs/throttler` (ordem e defaults)

Dois cuidados práticos:

- 1) **Ordem de inicialização** - O `ConfigModule` deve estar disponível antes do `ThrottlerModule.forRootAsync`.
- 2) **Defaults defensivos** - Em bootstrap, valores ausentes podem causar falhas. Portanto, forneça defaults coerentes.

Breaking change relevante: Throttler v6

Na versão 6, a configuração mudou: - Em vez de `{ ttl, limit }`, agora exige `throttlers: [{ ttl, limit }]`. - O TTL passou para **milissegundos**.

Exemplo didático:

```
// v6 (correto)
ThrottlerModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    throttlers: [
      {
        ttl: (Number(config.get('RATE_LIMIT_TTL')) ?? 60) * 1000,
        limit: Number(config.get('RATE_LIMIT_LIMIT')) ?? 100,
      },
    ],
  })
})
```

```
}),  
});
```

Aprendizado: bibliotecas evoluem; mudanças de versão podem quebrar bootstrap. Por isso, validação, defaults e documentação são parte do trabalho.

7.2) `migrationsRun: true` não é prática segura em produção

Para aprendizado e ambientes controlados, pode ajudar. Em produção, a recomendação é:

- Executar migrations via **script/CI/entrypoint controlado**, em uma ordem operacional segura: 1) rodar migrations; 2) executar smoke tests mínimos; 3) iniciar a aplicação.

Por quê? - Migrations longas podem segurar locks e causar downtime. - Execução automática no startup pode criar comportamentos não determinísticos em *rollout*.

7.3) Estratégia incremental (POC antes de propagar)

Para estudantes (e para times reais), a recomendação é:

1) aplicar `ConfigModule` + Throttler em **um serviço POC**; 2) validar boot, health e rotas principais; 3) só então propagar para os demais.

Rollback prático (conceito)

Documente sempre como reverter: - desabilitar temporariamente Throttler; - voltar para config anterior; - ou fixar versões de dependências.

8) Correções adicionais de 13/12/2025 — lições operacionais essenciais

As correções abaixo são altamente pertinentes porque representam erros comuns em projetos containerizados e com Postgres.

8.1) Defaults corretos para ambiente containerizado

- Em container, o host de DB normalmente é o nome do serviço (ex.: `db`), não `localhost`.
- Defaults errados em data-source causam falhas recorrentes.

8.2) `search_path` e extensão `citext`

- Extensões como `citext` ficam no schema `public`.
- Se o `search_path` não incluir `public`, pode falhar o acesso à extensão.

Regra prática: quando usar schema específico (ex.: `events`, `auth`) e também extensões no `public`, incluir ambos no `search_path`.

8.3) TypeORM ausente no registrations-service

- Mesmo com migrations e data-source, o módulo precisa importar o TypeORM.
- Padronizar a configuração evita “serviço fora do padrão”.

8.4) Compose consistente com `.env`

- Hardcode de credenciais em dev e env vars em prod cria inconsistência.
- Padronização com `${DB_USER}`, `${DB_PASS}`, `${DB_NAME}` reduz divergência.

8.5) Volumes persistentes e scripts de `postgres-init/`

- Scripts de init do Postgres **só rodam no primeiro start** (volume vazio).
- Se o volume já existe, mudar `.env` pode não surtir efeito.

Prática para dev (quando permitido perder dados):

```
docker compose down -v
```

8.6) Seed em `onModuleInit` deve ser defensivo

- Seeding antes das migrations pode quebrar o serviço.
- Em vez de crash, usar `try/catch` e log de aviso.

8.7) registrations-service ausente no compose de produção

- Deploy incompleto pode ocorrer quando o serviço não está listado.
- Checklist de deploy deve incluir todos os serviços esperados.

9) Pontos avançados (apenas para leitura e visão de evolução)

Estes pontos são relevantes, mas podem ser tratados como “próximas etapas” em uma turma:

9.1) Storage externo para rate limit

- Trocar in-memory por storage compartilhado (ex.: Redis) para suportar múltiplas réplicas.

9.2) Logs/métricas das tentativas

- Logar retries/limites atingidos e exportar métricas.

9.3) Node 18 vs Node 20

- Algumas dependências modernas esperam APIs globais presentes no Node 20.
- A solução definitiva é padronizar runtime (containers) para versões suportadas.

9.4) Build “limpo” e caches

- Em projetos TS com build incremental, é útil ter scripts de limpeza (`clean`) para evitar artefatos antigos.

10) Checklist final para alunos aplicarem no próprio projeto

1. Existe `ConfigModule` global com schema Joi?
 2. Variáveis críticas (DB, JWT, SERVICE_TOKEN, URLs internas) são obrigatórias?
 3. O serviço falha no startup se faltar env crítica?
 4. `helmet` está ativo?
 5. CORS está habilitado e parametrizado por env (origins) quando necessário?
 6. Rate limit está ativo (Throttler/guard) com defaults defensivos?
 7. `search_path` inclui schema do serviço e `public` quando houver extensões?
 8. Se há seeding automático, ele não derruba o serviço sem migrations?
 9. Compose usa env vars padronizadas e está documentado sobre volumes?
-

11) Conclusão

Este caso demonstra que “funcionar” não é o mesmo que “ser operável”. Em microsserviços, **configuração validada** e **segurança mínima** são requisitos básicos para confiabilidade.

Ao adotar `ConfigModule` + Joi, padronizar leitura via `ConfigService`, ativar `helmet` /CORS/rate limit e documentar os cuidados operacionais (volumes, `search_path`, migrations), o projeto ganha previsibilidade — e os alunos aprendem práticas que se aplicam diretamente ao mercado.